Journal of Scheduling (2024) 27:607–622
https://doi.org/10.1007/s10951-024-00821-0

Investigating constraint programming and hybrid methods for real world industrial test laboratory scheduling

Tobias Geibinger[1] · **Florian Mischek**[1] · **Nysret Musliu**[1]

Accepted: 3 August 2024 / Published online: 14 October 2024
© The Author(s) 2024

Abstract
In this paper we deal with a complex real world scheduling problem closely related to the well-known Resource-Constrained Project Scheduling Problem (RCPSP). The problem concerns industrial test laboratories in which a large number of tests are performed by qualified personnel using specialised equipment, while respecting deadlines and other constraints. We present different constraint programming models and search strategies for this problem. Furthermore, we propose a Very Large Neighborhood Search approach based on our CP methods. Our models are evaluated using CP solvers and a MIP solver both on real-world test laboratory data and on a set of generated instances of different sizes based on the real-world data. Further, we compare the exact approaches with VLNS and a Simulated Annealing heuristic. We could find feasible solutions for all instances and several optimal solutions and we show that using VLNS we can improve upon the results of the other approaches.

Keywords RCPSP · Resource constrained project scheduling problem · Project scheduling · Constraint programming · VLNS · Very large neighborhood search

1 Introduction

Project scheduling includes various problems of high practical relevance. Such problems arise in many areas and include different constraints and objectives. Usually project scheduling problems require scheduling of a set of project activities over a period of time and assignment of resources to these activities. Typical constraints include time windows for activities, precedence constraints between them, assignment of appropriate resources etc. The aim is to find feasible schedules that optimize several criteria such as the minimization of total completion time.

In this paper we investigate solving a real-world project scheduling problem that arises in an industrial test laboratory of a large company. This problem, Industrial Test Laboratory Scheduling (TLSP), which is an extension of the well known Resource-Constrained Project Scheduling Problem (RCPSP), was originally described by Mischek and Musliu (2018a, b, 2021). It consists of a grouping stage, where smaller activities (*tasks*) are joined into larger *jobs*, and a scheduling stage, where those jobs are scheduled and have resources assigned to them. In this work, we deal with the second stage and assume that a grouping of tasks into jobs is already provided. Since we focus on the scheduling part, we denote the resulting problem TLSP-S.

A note on terminology: Different terms have been used in the literature to denote the units of work in project scheduling problems that should be scheduled: Jobs, tasks, projects, batches, and others. In this paper we follow the convention by Brucker et al. (1999) and use the term *activities* when referencing general scheduling concepts or other project scheduling problems. Due to the specific distinction between tasks and jobs in TLSP(-S), we use these terms exclusively when referring to the corresponding TLSP(-S) concepts.

The investigated problem has several features of previous project scheduling problems in the literature, but also includes some specific features imposed by the real-world situation, which have rarely been studied before. Among

✉ Tobias Geibinger
tgeibing@dbai.tuwien.ac.at

Florian Mischek
fmischek@dbai.tuwien.ac.at

Nysret Musliu
musliu@dbai.tuwien.ac.at

[1] Christian Doppler Laboratory for Artificial Intelligence and Optimization for Planning and Scheduling, Databases and Artificial Intelligence Group, TU Wien, Favoritenstraße 9-11, Vienna 1040, Austria

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

others, these include heterogeneous resources, with availability restrictions on the activities each unit of a resource can perform. While work using similar restrictions exists (Dauzère-Pérès et al., 1998; Young et al., 2017), most problem formulations either assume homogeneous, identical units of each resource or introduce additional activity modes for each feasible assignment, which quickly becomes impractical for higher resource requirements and multiple resources. Another specific feature of TLSP(-S) is that of linked activities, which require identical assignments on a subset of the resources. To the best of our knowledge, a similar concept appears only in Salewski et al. (1997), where modes should be identical over subsets of all activities. We also deal with several non-standard objectives which arise from various business objectives of our industrial partner. Most notably, we try to minimize the duration of each project, i.e. the time between the start of the first and the end of the last job in the project, instead of the usual makespan minimization, which is counted from the fixed release date of the project's first activity.

In practice, exact solutions for this problem are desired especially in situations where it is necessary to check if a feasible solution exists at all. In the application that we consider, checking quickly if activities of additional projects can be added on top of an existing schedule is very important. In this paper we investigate exact methods for solving this problem. Although it is known from previous papers (Szeredi & Schutt, 2016; Young et al., 2017) that constraint programming techniques can give good results for similar project scheduling problems, it is an interesting question if Constraint Programming (CP) techniques can also solve TLSP-S that includes additional features and larger instances.

The current work is an extension of our previous conference paper (Geibinger et al., 2019). In particular, we introduce a new Very Large Neighborhood Search (VLNS) heuristic based on the CP models and show that this hybrid approach leads to improvements in solution quality. In addition, we provide a formal description of our CP model for the CP optimizer, and significantly extend our experimental section: We compare our results with those of a search heuristic that uses Simulated Annealing and apply our methods to a dataset taken from a real-life test laboratory.

The main contributions of this article are as follows:

- We provide two different CP models for our problem by exploiting some previous ideas for a similar problem from Szeredi and Schutt (2016) and Young et al. (2017) and extending them to model the additional features of TLSP-S. This includes, for example, the handling of the problem specific differences discussed above but also new redundant constraints as well as search procedures tailored to the problem. Using the MiniZinc (Nethercote et al., 2007) constraint programming language we experiment with various strategies involving the formulation of resource constraints, the reduction of the search space, and search procedures based on heuristics. The second model is geared towards the IBM CP Optimizer (IBM & CPLEX, 2017a) and leverages several scheduling-specific features of that solver.
- We propose a new hybrid VLNS-based approach that internally makes use of the CP models. This heuristic iteratively fixes a part of the schedule and solves the resulting (smaller) instance to optimality. It also includes various strategies to compute lower bounds and avoid redundant computations or local optima.
- We evaluate our approaches both on real-world data from an industrial test laboratory and randomly generated instances that are based on the real-world data. Our experiments show that constraint programming techniques can reach good results for realistic instances and outperform MIP solvers on the same model. Our results strengthen the conclusion of previous studies and show that CP technology can be applied successfully for solving large project scheduling problems. We also show that the hybridization of CP formulations with heuristics in the form of our VLNS approach is very successful in finding good solutions on instances of all sizes and further improves both upon the results of the CP models on the larger instances and those of a search heuristic based on Simulated Annealing.

The rest of the paper is organised as follows. In the next Section we give the related work. Section 3 introduces the problem that we investigate in this paper. A constraint model is given in Sect. 4, together with the alternative formulation for CP Optimizer. Section 5 contains a description of the VLNS procedure. Experimental results are presented in Sect. 6 and the last section gives conclusions.

2 Literature overview

The Resource-Constrained Project Scheduling Problem (RCPSP) has been investigated by numerous researchers over the last decades. For a comprehensive overview over publications dealing with this problem and its many variants, we refer to surveys e.g. by Brucker et al. (1999), Hartmann and Briskorn (2010), or Mika et al. (2015).

Of particular interest for the problem treated in this work are various extensions to the classical RCPSP.

Multi-Mode RCPSP (MRCPSP) formulations allow for activities that can be scheduled in one of several modes. This variant has been extensively studied since 1977 (Elmaghraby, 1977), we refer to the surveys by Wcglarz et al. (2011) and Hartmann and Briskorn (2010). A good example of a CP-

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

Model for the MRCPSP was given by Szeredi and Schutt (2016).

Many formulations, including TLSP, make use of release dates, due dates, deadlines, or combinations of those. An example of this can be found in a paper by Drezet and Billaut (2008). Further relevant extensions deal with multi-project formulations, including alternative objective functions (Pritsker et al., 1969). Usually, the objective in (variants of) RCPSP is the minimization of the total makespan (Hartmann & Briskorn, 2010). However, also other objective values have been considered. Of particular relevance to TLSP are objectives based on project durations and multi-objective formulations (both appear in e.g. Nudtasomboon and Randhawa (1997)). Salewski et al. (1997) include constraints that require several activities to be performed in the same mode. This is similar to the concept of linked jobs introduced in the TLSP.

RCPSP itself and most variants assume that individual units of each resource are identical and interchangeable. A problem closely related to TLSP-S is Multi-Skill RCPSP (MSPSP), first introduced by Bellenguez and Néron (2005). In this problem, each resource unit possesses certain skills, and an activity can only have those resources with the required skills assigned to it. This is similar to the availability restrictions on resources that appear in TLSP. Just like for our problem, they also deal with the problem that while availability restrictions could be modeled via additional activity modes corresponding to feasible resource assignments (Bartels & Zimmermann, 2009; Pritsker et al., 1969; Schwindt & Trautmann, 2000), this is intractable due to the large number of modes that would have to be generated (Bellenguez & Néron, 2005). To the best of our knowledge, the best results for the MSPSP problem have been achieved by Young et al. (2017), who use a CP model to solve the problem. We will be comparing and contrasting our CP model for TLSP-S with their model for MSPSP throughout this paper.

Also variants of VLNS have been used to solve the RCPSP or extensions of it, such as by Palpant et al. (2004). One of the main challenges in these approaches is the choice of a subset of activities that is selected for optimization at each step. In TLSP-S, we have multiple projects over which most of the constraints are evaluated. This makes single or multiple projects a natural choice for this subset, which we exploit in our heuristic.

Bartels and Zimmermann (2009) describe a problem for scheduling tests of experimental vehicles. It contains several constraints that also appear in similar form in TLSP-S, but includes a different resource model and minimises the number of experimental vehicles used.

3 Problem description

As mentioned before, we deal with a variant of TLSP (Mischek & Musliu, 2021), where we assume that a grouping of tasks into jobs is already provided for each project, and focus on the scheduling part of the problem instead (TLSP-S). Thus, the goal is to find an assignment of a mode, time slot and resources to each (given) job, such that all constraints are fulfilled and the objective function is minimized.

In the following, we introduce the TLSP-S problem.

Each instance consists of a scheduling period of h discrete *time slots*. Further, it lists resources of different kinds:

- A set E of *Employees* (shorthand Em) who are qualified for different types of jobs.
- A set B of *workbenches* (Wb) with different facilities.
- Various auxiliary lab *equipment* (Eq) groups. Each group g represents a set G_g of similar devices. The set of all equipment groups is denoted G^*.

Further we have given the set of *projects* labeled $P = \{1, \ldots, |P|\}$, and the set of *jobs* to be scheduled $J = \{1, \ldots, |J|\}$. For a project p, the jobs of this project are given as $J_p \subseteq J$.

Each job j has several properties[1]:

- A time window, given via a *release date* α_j and a *deadline* ω_j. In addition, it has a *due date* $\bar{\omega}_j$, which is similar to the deadline, except that exceeding it is only a soft constraint violation.
- A set of *available modes* $M_j \subseteq M$, where M is the set of all modes.
- A *duration* d_{mj} for each available mode $m \in M_j$.
- The resource requirements for the job j:
  - The number of *required employees* r_m^{Em} depends on the mode $m \in M_j$. Each of these employees must be chosen from the set of *qualified employees* $E_j \subseteq E$. Additionally, there is also a set of *preferred employees* $E_j^{\mathrm{Pr}} \subseteq E_j$.
  - The number of *required workbenches* $r_j^{\mathrm{Wb}} \in \{0, 1\}$. If a workbench is required, it must be chosen from the *available workbenches* $B_j \subseteq B$.
  - For each equipment group $g \in G^*$, the job requires r_{gj}^{Eq} devices, which must be taken from the set of *available devices* $G_{gj} \subseteq G_g$ for the group.
- The *predecessors* $\mathcal{P}_j$ of the job, which must be completed before the job can start. Precedence relations will only occur between jobs of the same project.

[1] In TLSP, these are derived from the tasks contained within a job. Since we assume the distribution of tasks into jobs to be fixed, they can be given directly as part of the input for TLSP-S.

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

- *Linked jobs* L_j of this job. All linked jobs must be performed by the same employee(s). As before, such links only occur between jobs of the same project.
- Optionally, the job may contain *initial assignments*.
  - An *initial mode* $\dot{m}_j$.
  - An *initial starting time slot* $\dot{s}_j$.
  - *Initial resource assignments*: For each employee $e \in E$, the boolean parameter $\dot{a}_{ej}^{\mathrm{Em}}$ indicates whether the employee e is initially assigned to j. Analogously, $\dot{a}_{bj}^{\mathrm{Wb}}$ and $\dot{a}_{dj}^{\mathrm{Eq}}$ perform the same function for each workbench $b \in B$ and each device $d \in G_g$, $g \in G^*$, respectively.

  Some or all of these assignments may be present for any given job.

Out of all jobs, a subset are *started jobs* $J^{\mathrm{S}} \subseteq J$. A started job will always fulfill the following conditions:

- It must have a preassigned mode.
- Its start time must be set to 1.
- It must have initial resource assignments fulfilling all requirements.

The initial assignments of a started job must not be changed in the solution.

A complete description of all constraints of the original model can be found in Mischek and Musliu (2021). The hard and soft constraints that we consider for the TLSP-S will be described in the next section, where we will introduce the CP model.

The aim for this problem is to find an assignment of a mode, time slot and resources to each (given) job, such that all hard constraints are fulfilled and the violation of soft constraints is minimized.

4 Constraint programming model

We developed our model using the solver-independent modeling language MiniZinc (Nethercote et al., 2007). Using MiniZinc we can easily compare different solvers. Furthermore, previous studies have shown that CP gives very good results for similar project scheduling problems. Most notably, the approaches by Young et al. (2017) and Szeredi and Schutt (2016) for MSPSP and MRCPSP respectively. MiniZinc also enables the use of user defined search strategies, which were shown to be very effective for MSPSP (Young et al., 2017). For both scheduling problems, the LCG solver Chuffed (Chu, 2011) was able to achieve very good results.

In order to provide an additional comparison, we also modeled our problem with the IBM ILOG CP Optimizer (IBM & CPLEX, 2017a). This model uses a different formulation of constraints and decision variables than the MiniZinc model and is described in Sect. 4.6.

A solution for the scheduling problem is represented by the following decision variables: The start time variable s_j assigns a start time to each job j. Similarly, for each job j, mode variable m_j assigns it a mode. For resource assignments we need the following variables: For each job j, the variable a_{ej}^{Em} is set to 1 if employee e is assigned to j and 0 otherwise, the variable a_{bj}^{Wb} is 1 if j is performed on workbench b and 0 otherwise, and the variable a_{dj}^{Eq} is 1 if device d is used by j and 0 otherwise. This representation of the resource assignments will prove to be useful for modelling the unary resource requirements in Sect. 4.2.

4.1 Basic hard constraints

The following constraints follow directly from the problem definition. As usual, the logical connective "and" (i.e. conjunction) is denoted by $\wedge$ and $\rightarrow$ stands for implication.

$$s_j \geq \alpha_j \ \wedge \ (s_j + d_{m_j j}) \leq \omega_j \qquad j \in J \qquad (1)$$

$$s_j \geq (s_k + d_{m_k k}) \qquad j \in J,\ k \in \mathcal{P}_j \qquad (2)$$

$$m_j \in M_j \qquad j \in J \qquad (3)$$

$$a_{ej}^{\mathrm{Em}} = 1 \ \rightarrow \ e \in E_j \qquad j \in J,\ e \in E \qquad (4)$$

$$a_{bj}^{\mathrm{Wb}} = 1 \ \rightarrow \ b \in B_j \qquad j \in J,\ b \in B \qquad (5)$$

$$a_{dj}^{\mathrm{Eq}} = 1 \ \rightarrow \ d \in G_{gj} \qquad j \in J, g \in G^*, d \in G_g \qquad (6)$$

$$\sum_{e \in E} a_{ej}^{\mathrm{Em}} = r_{m_j}^{\mathrm{Em}} \qquad j \in J \qquad (7)$$

$$\sum_{b \in B} a_{bj}^{\mathrm{Wb}} = r_j^{\mathrm{Wb}} \qquad j \in J \qquad (8)$$

$$\sum_{d \in G_g} a_{dj}^{\mathrm{Eq}} = r_{gj}^{\mathrm{Eq}} \qquad j \in J,\ g \in G^* \qquad (9)$$

$$a_{ej}^{\mathrm{Em}} = a_{ek}^{\mathrm{Em}} \qquad j \in J,\ k \in L_j, e \in E \qquad (10)$$

$$s_j = 1 \qquad j \in J^{\mathrm{S}} \qquad (11)$$

Constraint (1) makes sure that each job is executed in its time window, (2) enforces that the prerequisite jobs of a job are always completed before it starts. Constraints (3–6) ensure that assigned modes, employees, workbenches, and devices are available for the respective job. In order to make sure that each job has exactly as many resources as required, we have constraints (7–9). Furthermore, we need constraint (10) to make sure that linked jobs are assigned to the same employees and constraint (11) to fix the start time of jobs which are already started.

The above set of constraints is however not enough to ensure a valid solution. Additionally, we have to consider constraints which enforce that no resource (employee, work-

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

bench, or equipment) is assigned to two or more jobs at the same time. Like it was the case with MSPSP (Young et al., 2017), the constraints used for modeling those *unary resource requirements* have a tremendous impact on the practicability of the model and in the next subsection we will present different options for modeling such constraints.

4.2 Unary resource constraints

We will now present three different approaches for modeling unary resource constraints, each of which is designed with CP solvers in mind. Two of those three quickly proved to be impractical for our problem.

4.2.1 Time-indexed approach

The probably most straightforward way to model the non-overuse of any resource at any given time is captured by the following constraints, where the variable $f_j = s_j + d_{m_j j}$ captures the end of job j.

$$\sum_{j \in J, s_j \le t < f_j} a_{ej}^{\mathrm{Em}} \le 1 \qquad e \in E,\ 1 \le t \le h \tag{12}$$

$$\sum_{j \in J, s_j \le t < f_j} a_{bj}^{\mathrm{Wb}} \le 1 \qquad b \in B,\ 1 \le t \le h \tag{13}$$

$$\sum_{j \in J, s_j \le t < f_j} a_{dj}^{\mathrm{Eq}} \le 1 \qquad g \in G^*, d \in G_g, 1 \le t \le h \tag{14}$$

The number of constraints generated by MiniZinc based on (12–14) is of course directly dependent on the planning horizon h and the total number of resources. To further illustrate this, Fig. 1 shows the exemplatory source for the encoding of Constraint 12 in MiniZinc. During compilation, MiniZinc has to unravel the all the quantifiers including the sum, which means the number of generated constraints is cubic.

Because of the long compilation time and the high computer resource consumption, it quickly became clear that for our larger instances the time-indexed approach is not efficient (The model did not compile within our experimental time out, even for the smallest instances). This is of course not surprising since Young et al. (2017) came to a similar conclusion for MSPSP. Hence, we discarded this option after some preliminary testing.

4.2.2 Overlap constraint

For MSPSP, Young et al. (2017) achieved their best results using a so-called *order constraint*. This constraint basically enforces that two activities cannot overlap in their execution when they use a common resource. During the initial modeling phase we tried a very similar approach. First, we introduced the new predicate `overlap`:

$$\texttt{overlap}(j,k) := s_k < (s_j + d_{m_j j}) \ \wedge\ (s_k + d_{m_k k}) > s_j$$

In MSPSP, resources are assigned to activities with respect to the needed skill of the activity. For the overlap constraint it is not important which skill requirement the resource contributes to, so Young et al. (2017) had to introduce an auxiliary variable to express that a resource is used by an activity. We on the other hand assign the resources directly and thus can model our *overlap constraint* without any new variables.

$$\begin{aligned}\texttt{overlap}(j,k) \rightarrow (\ & \bigwedge_{e \in E} (\neg a_{ej}^{\mathrm{Em}} \vee \neg a_{ek}^{\mathrm{Em}}) \ \wedge \\ & \bigwedge_{b \in B} (\neg a_{bj}^{\mathrm{Wb}} \vee \neg a_{bk}^{\mathrm{Wb}}) \ \wedge \\ & \bigwedge_{g \in G^*, d \in G_g} (\neg a_{dj}^{\mathrm{Eq}} \vee \neg a_{dk}^{\mathrm{Eq}})\ ) \\ & j,k \in J,\ j \ne k,\ \alpha_k < \omega_j \wedge \omega_k > \alpha_j\end{aligned} \tag{15}$$

Just like with the time-indexed approach, it turned out that the overlap constraint produced too many constraints and was thus impractical for larger instances. This is interesting because Young et al. (2017) had no such problems, but their biggest instances only had 60 resources and 42 activities, whereas we have instances with more than 300 resources and jobs, respectively. It should however be noted that Young et al. (2017) reduced the number of generated constraints by considering only *unrelated* activity pairs, i.e. activities which do not depend on the execution of each other via precedence constraints (related activities can obviously never overlap). We on the other hand generate constraints for all pairs of jobs which are allowed to overlap based on their release dates and deadlines. Comparing only unrelated jobs requires the computation of the transitive closure of the job precedence relation and because both our generated instances and the real-world instances have a lot of unrelated jobs, we don't expect any significant improvement.

4.2.3 Cumulative constraints

Another way to model the unary resource constraints is to use `disjunctive` or `cumulative` global constraints. We have chosen the latter for our model, since none of the solvers we use in our experiments directly implement `disjunctive`. The `cumulative` constraint takes as input the start times, durations and resource requirements of a list of jobs and ensures that their resource assignments never exceed a given bound. This is of course a perfect way to enforce non overload of any resource and both MSPSP and MRCPSP have efficient models which make use of

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

```
constraint forall(e in Employees)(
  forall(t in TimeSlots)(
    sum(j in Jobs where startTime[j] <= t /\ t < (startTime[j] + duration[modeAssignment[j],j]))(
      employeeAssignment[e,j]
    ) <= 1
  )
);
```

Fig. 1 MiniZinc source code for Constraint (12)

`cumulative` in some way (Szeredi & Schutt, 2016; Young et al., 2017). Formally, a cumulative constraint is of the form `cumulative`(S, D, R, b) where S, D, and R are n-tuples of integer decision variables and b is an integer constant. To illustrate the semantics of the constraint suppose we have n activities with start times $S = \langle s_1, \ldots, s_n \rangle$, durations $D = \langle d_1, \ldots, d_n \rangle$, and resource usages $R = \langle r_1, \ldots, r_n \rangle$. Let $\mathcal{A}_t = \{i \in \mathbb{N}^+ \mid i \leq n,\ s_i \leq t \leq s_i + d_i\}$ be the set of activities active at time point t. We say that the cumulative constraint above is satisfied, if for every time point t it holds that $\sum_{i \in \mathcal{A}_t} r_i \leq b$.

In order to enforce the non-overload of any resource we need three constraints (one for each resource type).

$$\texttt{cumulative}((s_j)_{j\in J},\ (d_{m_j j})_{j\in J},\ (a^{\mathrm{Em}}_{ej})_{j\in J},\ 1) \qquad e \in E \tag{16}$$

$$\texttt{cumulative}((s_j)_{j\in J},\ (d_{m_j j})_{j\in J},\ (a^{\mathrm{Wb}}_{bj})_{j\in J},\ 1) \qquad b \in B \tag{17}$$

$$\texttt{cumulative}((s_j)_{j\in J},\ (d_{m_j j})_{j\in J},\ (a^{\mathrm{Eq}}_{dj})_{j\in J},\ 1) \qquad g \in G^*, d \in G_g \tag{18}$$

In contrast to our first two modeling approaches, this one turned out to scale well. Since the others performed so poorly on large instances, the rest of our experiments were performed with the `cumulative` unary resource constraints.

4.3 Soft constraints

There are several soft constraints in our problem definition (Mischek & Musliu, 2018b).

MiniZinc has no direct support for soft constraints, hence we define them as sums which should be minimised. Those sums are given as follows.

The first soft constraint depends on the number of jobs. Since we consider the job grouping fixed in TLSP-S, it is reduced to a constant value. In order to achieve comparability with results for TLSP, we keep it as a constant term $s_1 = w_1 \cdot |J|$ in the objective value.

We give preference to solutions in which the assigned employees of a job are taken from the set of preferred employees:

$$s_2 = w_2 \cdot \sum_{j\in J} \sum_{e\in(E\setminus E^{\mathrm{Pr}}_j)} a^{\mathrm{Em}}_{ej}$$

For each project, the total number of employees assigned to it should be minimised:

$$s_3 = w_3 \cdot \sum_{p\in P} \sum_{e\in E} ((\sum_{j\in J_p} a^{\mathrm{Em}}_{ej}) > 0)$$

For each job, violating its due date should be avoided:

$$s_4 = w_4 \cdot \sum_{j\in J} max(s_j + d_{m_j j} - \bar{\omega}_j, 0)$$

Lastly, project durations should be as small as possible:

$$s_5 = w_5 \cdot \sum_{p\in P} (max_{j\in J_p}(s_j + d_{m_j j}) - min_{j\in J_p}(s_j))$$

The objective of the search is then given by

$$min \sum_{2\leq i\leq 5} s_i.$$

At the moment, the values of the weights w_i $(1 \leq i \leq 5)$ are being determined in correspondence with a real-world laboratory. Currently all these weights are set to 1.

4.4 Redundant constraints

While the formulation of the constraints given above correctly represents the problem, we also study redundant constraints to enhance solver performance. Such constraints allow CP solvers to propagate decisions in additional ways, potentially leading to conflicts earlier and thus pruning the search space.

Finding good redundant constraints for our problem proved to be very hard since the search space is usually very big and at the beginning of the search there is little knowledge about the final duration of the jobs. To deal with this issue we introduced a relaxed `cumulative` constraint enforcing

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

a global resource bound.

$$\texttt{cumulative}((s_j)_{j\in J}, (min_{m\in M_j}(d_{mj}))_{j\in J}, (min_{m\in M_j}(r_m^{\text{Em}}) + r_j^{\text{Wb}} + \sum_{g\in G^*} r_{gj}^{\text{Eq}})_{j\in J}, |E| + |B| + \sum_{g\in G^*} |G_g|) \quad (19)$$

This enables the search to discard scheduling options which are impossible regardless of the chosen modes early on.

On top of that, we can also formulate more straightforward `cumulative` constraints which enforce the global resource bounds for each resource at any point in time.

$$\texttt{cumulative}((s_j)_{j\in J}, (d_{m_j j})_{j\in J}, (r_{m_j}^{\text{Em}})_{j\in J}, |E|) \quad (20)$$

$$\texttt{cumulative}((s_j)_{j\in J}, (d_{m_j j})_{j\in J}, (r_j^{\text{Wb}})_{j\in J}, |B|) \quad (21)$$

$$\texttt{cumulative}((s_j)_{j\in J}, (d_{m_j j})_{j\in J}, (r_{gj}^{\text{Eq}})_{j\in J}, |G|) \quad g \in G^* \quad (22)$$

Given the large search space, trying to restrict the scope of the decision variables seems like a worthwhile idea. We achieve this by using *global cardinality constraints*. There are a number of global cardinality constraints in MiniZinc but we use ones of the form $\texttt{gcc_low_up}(V, c, l, u)$ where $V = \langle v_1, \ldots, v_n\rangle$ is an n-tuple of decision variables, c is a domain element, and l, u are constant non-negative integers. Let $o = |\{i \in \mathbb{N}^+ \mid i \leq n,\ v_i = c\}|$ be the number of occurrences of the domain value c in the assignment of variables V, then the constraint is satisfied if $l \leq o \leq u$.

The following constraints allow us to set tight bounds for the total number of resources which should be used.

$$\texttt{gcc_low_up}\left((a_{ej}^{\text{Em}})_{e\in E, j\in J},\ 1,\ \sum_{j\in J} min_{m\in M_j}(r_m^{\text{Em}}),\ \sum_{j\in J} max_{m\in M_j}(r_m^{\text{Em}})\right) \quad (23)$$

$$\texttt{gcc_low_up}\left(\left(a_{bj}^{\text{Wb}}\right)_{b\in B, j\in J},\ 1, \sum_{j\in J} r_j^{\text{Wb}}, \sum_{j\in J} r_j^{\text{Wb}}\right) \quad (24)$$

$$\texttt{gcc_low_up}\left(\left(a_{dj}^{\text{Eq}}\right)_{g\in G^*, d\in G_g, j\in J},\ 1, \sum_{j\in J}\sum_{g\in G^*} r_{gj}^{\text{Eq}}, \sum_{j\in J}\sum_{g\in G^*} r_{gj}^{\text{Eq}}\right) \quad (25)$$

Constraint (23) enforces that no more employees can be assigned than the sum of the highest possible employee requirements and no less than the sum of the minimum requirements. The other two constraints analogously ensure that the number of assigned workbenches and equipment is tightly bounded by the cumulative requirement of all jobs.

4.5 Search strategies

During initial testing results immediately showed that the default search strategy of Chuffed (or Gecode) was not even able to find feasible solutions for most instances. This was not surprising since Young et al. (2017) already had a similar issue with MSPSP. However, they were able to improve their results drastically by employing a new MiniZinc search annotation called `priority_search` which is supported by Chuffed (Feydy et al., 2017). In general, search annotations in MiniZinc allow us to influence the search by modifying the value or variable selection strategies of the underlying solver. The relatively new `priority_search` enables the specification of more complex selection strategies which are not definable with other annotations. For example, it allows us to tell the solver that whenever a decision is to be made, then it should make assignments for the unassigned job with the smallest possible start time.

Based on the research of Young et al. (2017), we have experimented with four slightly different versions of `priority_search`:

(i) `ps_startFirst_aff`
(ii) `ps_startFirst_ff`
(iii) `ps_modeFirst_aff`
(iv) `ps_modeFirst_ff`

All four search strategies branch over the jobs and their resource assignments. The order of the branching is the same for all strategies and is determined by the smallest possible start times of the jobs in ascending order. For each branch, searches (i) and (ii) initially assign the smallest start time to the selected job followed by assigning it the mode which minimises the job duration. Search procedures (iii) and (iv) start with the mode assignment and then assign the start time. Once the start time and the mode have been assigned for the selected job, all of the search strategies make resource assignments for the job. Searches (i) and (iii) start by assigning those resources to the job which are available and have the biggest

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

domain, whereas (ii) and (iv) start with assignments which are either unavailable or have only one value in their domain.

4.6 Alternative CP model

We also modeled our problem with CP Optimizer (IBM & CPLEX, 2017a).

In this model, all the decision variables are (possibly optional) intervals. Optional intervals are not necessarily present in a solution unless some constraints require it. Intervals which are not present are ignored by other constraints. Internally, intervals contain a start and an end time and when declaring an interval we can directly constrain their domains. The keywords `startOf`(a) and `endOf`(a) can be used to obtain the start and respectively end of an interval a and `lengthOf`(a) returns its length. Furthermore, it is also possible to fix the size of an interval variable at declaration. Furthermore, the constraint `noOverlap`(I), where I is a tuple of intervals, ensures that only the present intervals of I do not overlap. The presence (or non-presence) of an interval a can be enforced using the Boolean function `presenceOf`(a) which is true whenever a is present and an appropriate constraint. The constraint `alternative`(a, I, n), where a is an interval, I a tuple of intervals and n is a natural number, can also affect the presence of intervals. This constraint is true if whenever a is present, also exactly n intervals from I are present as well. Further CP Optimizer constraints we use are `endBeforeStart`(a_1, a_2) enforcing that interval a_1 ends before the start of interval a_2, and `span`(a, I) which constrains the interval a to start at the earliest start and end at the latest end of any interval in I.

For a complete formal definition of the CP Optimizer language and all the supported constraints, we refer to Laborie et al. (2018).

In this model the decision variables are given by the following interval variables:

$$\texttt{interval } a_j \subset [\alpha_j, \omega_j) \qquad j \in J$$
$$\texttt{interval } b_p \qquad p \in P$$
$$\texttt{interval } a_{ij} \texttt{ optional size } d_{ij} \qquad j \in J, i \in M_j$$
$$\texttt{interval } a^{\mathrm{EmM}}_{eij} \texttt{ optional} \qquad j \in J, i \in M_j, e \in E_j$$
$$\texttt{interval } a^{\mathrm{Em}}_{ej} \texttt{ optional} \qquad j \in J, e \in E_j$$
$$\texttt{interval } a^{\mathrm{Wb}}_{bj} \texttt{ optional} \qquad j \in J, b \in B_j$$
$$\texttt{interval } a^{\mathrm{Eq}}_{gdj} \texttt{ optional} \qquad j \in J, g \in G^*, d \in G_{gj}$$

The intervals a_j represents the jobs and are constrained to the time windows of the respective job. The second set of intervals b_p are auxiliary variables representing the total duration of the projects. Those intervals enable an easy formulation of the project duration soft constraint. The next intervals a_{ij} are optional and the presence of such an interval indicates that job j is performed in mode i. For a job j several employee allocations are possible depending on its mode. The presence of an optional interval a^{EmM}_{eij} represents the allocation of employee e to perform job j in mode i. The last three sets of optional intervals are used to indicate resource allocation.

The hard constraints of the problem are encoded with the following constraints:

$$\texttt{span}(b_p, [a_j]_{j \in J_p}) \qquad p \in P \quad (26)$$
$$\texttt{endBeforeStart}(a_k, a_j) \qquad k \in P_j \quad (27)$$
$$\texttt{alternative}(a_j, (a_{ij})_{i \in M_j}, 1) \qquad j \in J \quad (28)$$
$$\texttt{alternative}(a_{ij}, (a^{\mathrm{EmM}}_{eij})_{e \in E_j}, r^{\mathrm{Em}}_{ij}) \qquad j \in J, i \in M_j \quad (29)$$
$$\texttt{alternative}(a^{\mathrm{Em}}_{ej}, (a^{\mathrm{EmM}}_{eij})_{i \in M_j}, 1) \qquad j \in J, e \in E_j \quad (30)$$
$$\texttt{noOverlap}((a^{\mathrm{EmM}}_{eij})_{j \in J, i \in M_j : e \in E_j}) \qquad e \in E \quad (31)$$
$$\texttt{noOverlap}((a^{\mathrm{Em}}_{ej})_{j \in J : e \in E_j}) \qquad e \in E \quad (32)$$
$$\texttt{alternative}(a_j, (a^{\mathrm{Wb}}_{bj})_{b \in B_j}, r^{\mathrm{Wb}}_j) \qquad j \in J \quad (33)$$
$$\texttt{noOverlap}((a^{\mathrm{Wb}}_{bj})_{j \in J : b \in B_j}) \qquad b \in B \quad (34)$$
$$\texttt{alternative}(a_j, (a^{\mathrm{Eq}}_{gdj})_{d \in G_{gj}}, r^{\mathrm{Eq}}_{gj}) \qquad j \in J, g \in G^* \quad (35)$$
$$\texttt{noOverlap}((a^{\mathrm{Eq}}_{gdj})_{j \in J : d \in G_{gj}}) \qquad g \in G^*, d \in G_g \quad (36)$$
$$\neg\texttt{presenceOf}(a_{ji}) \qquad j \in J, i \in (M \setminus M_j) \quad (37)$$
$$\neg\texttt{presenceOf}(a^{\mathrm{Em}}_{ej}) \qquad j \in J, b \in (E \setminus E_j) \quad (38)$$
$$\neg\texttt{presenceOf}(a^{\mathrm{Wb}}_{bj}) \qquad j \in J, b \in (B \setminus B_j) \quad (39)$$
$$\neg\texttt{presenceOf}(a^{\mathrm{Eq}}_{dj}) \qquad j \in J, g \in G^*, d \in (G_g \setminus G_{gj}) \quad (40)$$
$$\texttt{presenceOf}(a^{\mathrm{Em}}_{ej}) = \texttt{presenceOf}(a^{Em}_{ek}) \qquad j \in J, k \in L_i, e \in E \quad (41)$$
$$\texttt{startOf}(a_j) = 1 \qquad j \in J^{\mathrm{S}} \quad (42)$$

Constraint (26) allows us to easily formulate the soft constraints which depend on the length of a project. The job precedences are enforced through Constraint (27). The alternative constraints (29–31) make sure that each job is assigned enough employees for the selected mode, while (31) and (32) express that no employee can be assigned to two jobs at the same time. The constraints (33–36) enforce the same for workbenches and equipment. In order to make sure that only available resources are assigned to the jobs, we need constraints (37–40). Lastly, linked jobs are modeled using (41) and started jobs with (42).

We can also formulate `cumulative` constraints in CP Optimizer in the following way: $\sum_{i \leq n} \texttt{pulse}(a_i, r_i) \leq b$, where $a_i, \ldots, a_n$ are intervals, $r_i, \ldots, r_n$ their resource usages, and b is the resource bound. The following redun-

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

dant constraints are similar to the `cumulative` constraints defined in Sect. 4.4.

$$\sum_{j \in J} \sum_{m \in M_j} \texttt{pulse}(a_{jm}, r^{\text{Em}}_{jm}) \leq |E| \tag{43}$$

$$\sum_{j \in J} \sum_{m \in M_j} \texttt{pulse}(a_{jm}, r^{\text{Wb}}_{jm}) \leq |B| \tag{44}$$

$$\sum_{j \in J} \sum_{m \in M_j} \texttt{pulse}(a_{jm}, r^{\text{Eq}}_{jm}) \leq \sum_{g \in G^*} |G_g| \tag{45}$$

Finally, the objective is to minimize the following formula which is just the sum of all soft constraints defined in Sect. 4.3.

$$\begin{aligned} min \quad & w_2 \cdot \sum_{j \in J} \max(0, \texttt{endOf}(a_j) - \overline{\omega}_j) \\ & + w_3 \cdot \sum_{j \in J} \sum_{e \in (E \setminus E^{Pr}_j)} \texttt{presenceOf}(a^{\text{Em}}_{ej}) \\ & + w_4 \cdot \sum_{p \in P} \sum_{e \in E} \big(0 < \sum_{j \in J_p} \texttt{presenceOf}(a^{\text{Em}}_{ej})\big) \\ & + w_5 \cdot \sum_{p \in P} \texttt{lengthOf}(b_p) \end{aligned}$$

5 Very large neighborhood search

Utilising our CP model we implemented a *Very Large Neighborhood Search*. The basic idea is to start with a feasible but suboptimal solution and repeatedly fix most of the schedule except for a small number of projects and then try to find an optimal solution for the unfixed projects (each such intermediate optimization is a *move*).

The basic steps of the search are the following:

1. *Find initial solution* In order for the VLNS to work, we need a feasible schedule for the instance. Since our CP model approach finds feasible solutions fast, we used it to provide an initial solution. In particular, we use Chuffed with our MiniZinc CP model, `priority_search` and *free search* disabled. This approach usually yields feasible schedules within one minute, even for large instances.
2. *Calculate lower bound for each project* In parallel to step one we use our CP approach to determine a lower bound for each project. This is done by solving each project once without regard for the other projects. The optimal objective is then a lower bound for the penalty of the project in the original problem. Since this can take a long time for bigger projects we define a runtime limit of 30 s for each project. If the limit is reached and no optimal solution could be found, we determine a heuristic lower bound for the project as follows: We add up the number of jobs (s_1), the minimum number of different employees needed for the project (s_3), and the minimal duration of all jobs on the longest path in the job dependency graph (s_5). For s_2 and s_4, 0 is used as the lower bound.
3. *Fix all but k projects* Once we have an initial solution and the lower bounds we can start the actual heuristic. We start by selecting at random a combination of k projects (initially, $k = 1$), with the following properties: All of them overlap in the current schedule (or, if there are no such combinations, have overlapping time windows) and at least one of the projects has potential for improvement i.e. the difference between the current penalty and the lower bound is bigger than zero. The projects which are not in the selected combination are then fixed. To achieve this, we modify the time windows and availabilities of each job contained in a fixed project. The release and due dates are changed to correspond to the current start and end dates, the available modes and resources are changed to only include the current assignments, and finally the resource requirements are restricted to equal the assignments. The resulting instance is then further reduced by cutting away the fixed jobs outside of the merged time window of the selected projects. These removed jobs cannot be changed or influence the result and reducing the size of the instance improves compilation time.
4. *Perform move* After the preprocessing we try to find an optimal solution for the selected projects, where we again set a runtime limit of 30 s. The best assignment found within this limit is then applied to the current incumbent schedule if it does not increase the penalty of the solution. With a parameterised probability $hotStartProb$ we *hot start* the CP solver. This means that the current assignment of the selected projects is given to the solver as an initial solution. The hot start functionality is based on work by Demirovic et al. (2018) and has been integrated by us into the current version of the solver Chuffed. Hot starting the search however means that the search never accepts moves which change the schedule but do not decrease the penalty. This is why we only hot start with a given probability. If we do not hot start the solver, we additionally set up `priority_search` to assign resources not in input order but randomly. This further enhances diversification. After we have performed the move, we save the selected combination of the k projects in a list. Combinations contained in this list (or subsets of a combination in the list) are not selected again unless there has been a change in a job overlapping the contained projects.
5. *Possibly change k* If k is bigger than 1 and the incumbent schedule has been changed by the last move, then we set k back to 1. Otherwise, if there are no more eligible combinations with size k, we increase k by one or—with probability $jumpProb$—by two. If there no more combinations for any k, we terminate. After this we check if

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

the current solution is equal to the sum of all project lower bounds. Should that be the case, then we have found an optimal solution and can terminate. If not, we go back to step 3 or terminate if we have reached the time limit of the solver.

6 Experiments and comparison

For our experimental evalution, we used MiniZinc 2.2.3 (Nethercote et al., 2007) with Chuffed 0.10.3 (Chu, 2011) and CPLEX 12.8.0 (IBM & CPLEX, 2017b). Our VLNS which is described in Sect. 5 was implemented in Java 8. Furthermore, we also experimented with the ILOG CP Optimizer 12.8.0 (IBM & CPLEX, 2017a) which was not run from MiniZinc but with the ILOG Java API. We have also tested Gecode (Schulte et al., 2018) as an additional CP solver included in MiniZinc, but it quickly proved to be inferior to Chuffed on this model even when run with multiple threads. Regarding comparison to other approaches in the literature, to the best of our knowledge no solutions exist yet for the problem we consider in this paper.

The experiments in Sects. 6.2–6.4 were ran on a benchmark server with 224GB RAM and two AMD Opteron 6272 Processors each with max. 2.1GHz and 8 physical cores. Since all of the solvers we experimented with are single threaded, we usually ran multiple independent sets of benchmarks in parallel.

For Sect. 6.5 we employed a computational cluster with 13 nodes, each having 2 Intel Xeon CPUs E5-2650 v4 (max. 2.90GHz, 12 physical cores, no hyperthreading).

In addition to the solution approaches described in this paper, we also compare with a solution approach by Mischek and Musliu (2021) using Simulated Annealing (SA). SA (first described by Kirkpatrick et al. (1983)) is a well-known metaheuristic which works by repeatedly applying small random changes (moves) to a candidate solution. Improvements are always accepted, while worsening moves are accepted with a certain probability that depends on the difference in the objective function and a parameter called *temperature*. Higher temperatures lead to higher acceptance probabilities. Over the course of the search, the temperature is slowly decreased.

6.1 Instances

We use a total of 30 randomly generated instances (based on real-life situations) of different sizes for our experiments. Those instances are listed in Table 1.

While those instances were generated randomly, they are still modeled after real-world scenarios. To this end, we developed an instance generator that can create instance of different sizes and properties, while matching the distribution of durations and resources of the real-world data. The general idea is that a (feasible, but not necessarily optimal) reference solution is created first, which is then used as the basis for defining the properties of projects and jobs. That way, it is always guaranteed that each instance has a feasible solution. The details of how the instance generation works and what configurations are possible are described in (Mischek & Musliu, 2021).

For half of the instances the generator was configured to very closely match the real-world laboratory of our industrial partner (data set *LabStructure*). The other half of the instances (data set *General*) is more general and makes full use of various problem features, such as the number and design of equipment groups, the structure of the dependency graph of each project and others.

The instances all have three modes: a *single* mode requiring only one employee, a *shift* mode which requires two employees but has a reduced duration, and an *external* mode that requires no employees at all. In general, jobs can be done in single mode or optionally in shift mode. Some instances however also include jobs which can only be performed in external mode. Initial assignments appear only for jobs which are already started or are fixed to their current value via availability restrictions and time windows (this is the case for around 5% of all jobs).

Since the benchmark instances were generated with the full TLSP in mind and in TLSP-S we take the initial job grouping as fixed and unchangeable, the instances had to be converted. This is achieved by viewing the jobs as the smallest planning unit and assigning the job parameters—which are defined by the tasks contained in the job in TLSP—directly to the jobs.

The 30 benchmark instances are a selection from a total of 120 generated instances. We chose the first two instances of each size (scheduling horizon and number of projects) and two additional instances for the 3 smallest sizes. This selection was necessary because of the long time it would have taken to experiment with all 120 instances. In total, our benchmark data set contains instances which have 5 to 90 projects, between 7 and 401 jobs, and span 88 to 782 time slots.

In addition, we also include results for one real-life instance (Lab) taken directly from the lab of our industrial partner (in anonymized form). This instance covers a scheduling period of over one year, at a granularity of two timeslots per working day. It includes a reference schedule that is the manually created schedule actually used in the lab at the time the instance was created.

All instances can be downloaded from https://www.dbai.tuwien.ac.at/staff/fmischek/TLSP/.

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

Table 1 The set of test instances used for the experiments

#	Data Set	ID	$\|P\|$	$\|J\|$	h	$\|E\|$	$\|B\|$	$\|G^*\|$	$\overline{\|M_j\|}$	$\overline{\|E_j\|}$	$\overline{\|B_j\|}$	$\overline{\|G_{gj}\|}$
1	General	000	5	7	88	7	7	3	1.43	2.08	3.57	1.50
2	General	001	5	8	88	7	7	3	1.75	4.88	3.63	15.67
3	LabStructure	000	5	24	88	7	7	3	1.71	1.84	3.38	11.67
4	LabStructure	001	5	14	88	7	7	3	1.79	4.36	3.5	0.36
5	General	005	10	29	88	13	13	4	1.52	4.04	3.48	5.76
6	General	006	10	18	88	13	13	6	1.50	5.56	4.22	13.28
7	LabStructure	005	10	37	88	13	13	3	1.68	6.16	4.03	0.65
8	LabStructure	006	10	29	88	13	13	3	1.62	6.21	3.76	21.01
9	General	010	20	60	174	16	16	5	1.57	7.42	4.42	11.36
10	General	011	20	84	174	16	16	4	1.58	7.31	4.30	3.70
11	LabStructure	010	20	65	174	16	16	3	1.52	6.28	4.43	26.26
12	LabStructure	011	20	62	174	16	16	3	1.68	7.27	4.24	1.21
13	General	020	15	29	174	12	12	5	1.55	5.76	3.97	1.12
14	LabStructure	020	15	53	174	12	12	3	1.62	6.28	4.47	20.63
15	General	025	30	113	174	23	23	3	1.59	8.26	4.41	5.71
16	LabStructure	025	30	105	174	23	23	3	1.65	7.52	4.25	39.63
17	General	015	40	126	174	31	31	3	1.56	9.26	4.48	29.53
18	LabStructure	015	40	138	174	31	31	3	1.64	7.36	3.57	41.93
19	General	030	60	208	174	46	46	6	1.63	9.85	4.11	31.45
20	LabStructure	030	60	212	174	46	46	3	1.63	9.28	4.17	78.16
21	General	035	20	76	520	6	6	5	1.70	4.24	3.62	8.08
22	LabStructure	035	20	71	520	6	6	3	1.68	4.30	3.42	11.70
23	General	040	40	196	520	12	12	4	1.67	6.95	4.47	4.24
24	LabStructure	040	40	187	520	12	12	3	1.65	6.55	4.51	1.38
25	General	045	60	260	520	18	18	6	1.58	7.65	4.52	23.95
26	LabStructure	045	60	239	520	18	18	3	1.63	7.44	4.42	33.65
27	General	050	60	270	782	13	13	4	1.65	6.89	4.39	3.89
28	LabStructure	050	60	247	782	13	13	3	1.63	6.97	4.21	23.42
29	General	055	90	384	782	19	19	5	1.64	7.27	4.29	26.89
30	LabStructure	055	90	401	782	19	19	3	1.66	7.34	4.53	36.76
Lab	–	–	74	297	606	22	17	1	1.25	5.70	3.48	1

Shown are the data set the instance is taken from and the ID within that set. The following columns list the number of projects, jobs and the length of the scheduling period, followed by the number of employees, workbenches and equipment groups. The last columns contain the mean available modes, qualified employees and available workbenches per job, as well as the mean available devices per job and equipment group (only over those jobs that actually require at least one device of the group, about 10% of all jobs). *The discrepancy compared to the generated instances arises from the fact that several equipment groups were not yet considered for planning at the time this instance was created

6.2 Search and redundant constraints experiments

Table 2 shows the comparison of search procedures and redundant constraints described in Sects. 4.5 and 4.4 respectively. The column *#sat* shows for how many instances (out of the 30 generated instances) the model-search combination found feasible solutions, in difference *#opt* contains the number of instances solved to optimality. Furthermore, the values in the column *#best* show the number of times the best solutions (in this set of experiments) was found for an instance, and *avg rel. dist. to best* is the average relative distance to the best solution.

Each model was run using Chuffed with free search enabled. Free search alternates between user-defined and activity-based search on each restart. The time limit was set to 30 min for each instance. It can be easily seen that any version of `priority_search` is vastly superior to the default search of Chuffed. `priority_search` strategies solve more instances to optimality with any of the redundant constraints and 15 of the configurations using `priority_search` find a feasible solution for every

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

Table 2 Priority search and redundant constraints experiments (runtime 30 m)

Search	Constraints	#sat	#opt	#best	avg rel. dist. to best (%)
Default	(1–10), (16–18)	12	8	8	11.43
Default	(1–10), (16–18), (19)	13	8	8	14.1
Default	(1–10), (16–18), (19–22)	13	8	8	14.68
Default	(1–10), (16–18), (20–22)	13	8	8	14.04
Default	(1–10), (16–18), (20–25)	13	8	8	13.25
Default	(1–10), (16–18), (23–25)	12	8	8	12.99
Default	(1–10), (16–18), (19–25)	13	8	8	12.98
`ps_startFirst_aff`	(1–10), (16–18)	28	**14**	17	1.55
`ps_startFirst_aff`	(1–10), (16–18), (19)	28	**14**	17	1.54
`ps_startFirst_aff`	(1–10), (16–18), (19–22)	**30**	**14**	18	0.91
`ps_startFirst_aff`	(1–10), (16–18), (20–22)	28	**14**	17	1.55
`ps_startFirst_aff`	(1–10), (16–18), (20–25)	**30**	**14**	**22**	**0.66**
`ps_startFirst_aff`	(1–10), (16–18), (23–25)	28	**14**	20	1.59
`ps_startFirst_aff`	(1–10), (16–18), (19–25)	**30**	**14**	16	0.78
`ps_startFirst_ff`	(1–10), (16–18)	28	**14**	18	1.16
`ps_startFirst_ff`	(1–10), (16–18), (19)	28	**14**	17	1.8
`ps_startFirst_ff`	(1–10), (16–18), (19–22)	**30**	**14**	17	0.95
`ps_startFirst_ff`	(1–10), (16–18), (20–22)	**30**	**14**	18	0.91
`ps_startFirst_ff`	(1–10), (16–18), (20–25)	**30**	**14**	21	0.75
`ps_startFirst_ff`	(1–10), (16–18), (23–25)	28	**14**	19	1.7
`ps_startFirst_ff`	(1–10), (16–18), (19–25)	**30**	**14**	16	0.86
`ps_modeFirst_aff`	(1–10), (16–18)	28	**14**	15	3.31
`ps_modeFirst_aff`	(1–10), (16–18), (19)	28	**14**	14	2.4
`ps_modeFirst_aff`	(1–10), (16–18), (19–22)	**30**	**14**	14	2.81
`ps_modeFirst_aff`	(1–10), (16–18), (20–22)	**30**	**14**	14	2.68
`ps_modeFirst_aff`	(1–10), (16–18), (20–25)	**30**	**14**	15	2.7
`ps_modeFirst_aff`	(1–10), (16–18), (23–25)	28	**14**	15	2.84
`ps_modeFirst_aff`	(1–10), (16–18), (19–25)	**30**	**14**	14	2.95
`ps_modeFirst_ff`	(1–10), (16–18)	28	**14**	15	3.32
`ps_modeFirst_ff`	(1–10), (16–18), (19)	28	**14**	14	2.41
`ps_modeFirst_ff`	(1–10), (16–18), (19–22)	**30**	**14**	14	2.75
`ps_modeFirst_ff`	(1–10), (16–18), (20–22)	**30**	**14**	14	2.68
`ps_modeFirst_ff`	(1–10), (16–18), (20–25)	**30**	**14**	15	2.68
`ps_modeFirst_ff`	(1–10), (16–18), (23–25)	28	**14**	16	2.83
`ps_modeFirst_ff`	(1–10), (16–18), (19–25)	**30**	**14**	14	2.95

Bold values indicate the best result obtained in this comparison

instance. It should be noted that the fourteen optimally solved instances are the same over all configurations. They all have less than or equal to 20 projects. The search strategy `ps_startFirst_aff` in combination with redundant constraints (20–25) achieved the best results in general, which is why this strategy—albeit slightly modified to support random value selection—is also used in the VLNS.

Which redundant constraints are important, seems to depend on the used search strategy. However, all `priority_search` strategies achieve their best results using all redundant constraints but constraint (19). Furthermore, when comparing the configurations using no redundant constraints and the ones using only redundant constraint (19), there seems to be no significant improvement by adding the constraint except for the default search strategy. The effects of redundant constraints (23)-(25) are also interesting. Adding them exclusively appears to have negative effects for all search configurations, whereas in conjunction with constraints (20)–(22) they improve the results in almost all cases including the overall best configuration.

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

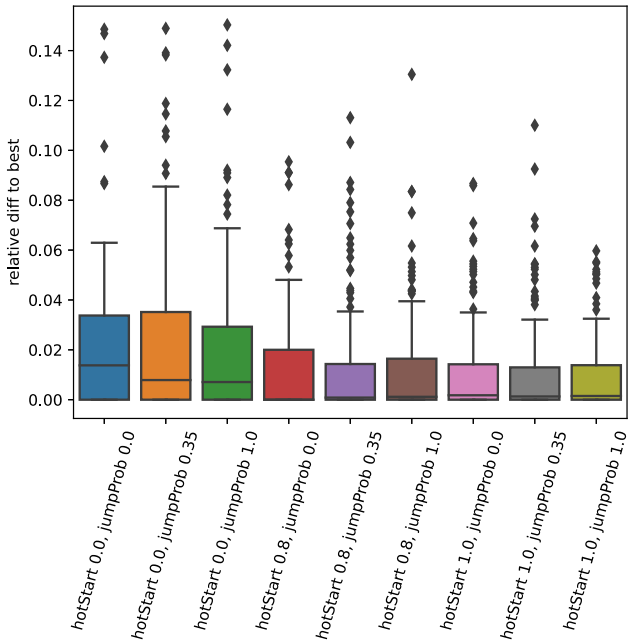

Fig. 2 Comparison of selected VLNS configurations

6.3 VLNS parameter configuration

As described in Sect. 5, we have 2 parameters for VLNS: There is the probability for hot starting a move called *hotStartProb* and there is the probability that if we have to change the neighborhood—i.e. the number of projects we modify in a single move—we increase it not by 1 but by 2. The latter parameter is called *jumpProb*.

We tested several different configurations of those parameters and one trend which was clearly distinguishable was that setting the *hotStartProb* to zero gives rather bad results. An explanation for this is that not hot starting moves may cause them to timeout before they find an improvement which wastes a lot of runtime. For the other configurations there were no significant performance differences between them, but hot starting 80% of the time and jumping 35% performed slightly better than the rest.

Figure 2 shows the results of our experiments for selected configurations. We ran each instance with each configuration five times and each data point in the plot represents the relative difference of a run to the best solution found for the instance. It can be seen that regardless of the *jumpProb*, the configurations where *hotStartProb* is zero are clearly the worst. However, the other configurations show a very similar performance.

6.4 Comparison

Table 3 shows our next experiment, which is a comparison between Simulated Annealing, VLNS, Chuffed, the MIP solver CPLEX and the CP solver CP Optimizer. The time limit was set to 2 h for each instance. CP Optimizer was run with 8 threads and the parameter *FailureDirectedSearchEmphasis* was set to 4. For Simulated Annealing we used the implementation of Mischek and Musliu (2021), with the same parameter configuration.

It can easily be seen that CPLEX performed very poorly in comparison to the rest, although it should be noted that our model was developed with CP solvers in mind and thus might not be a perfect fit for MIP solvers. Also, CPLEX was only run in single-threaded mode.

Chuffed on the contrary could find solutions for all instances and even found and proved optimal solutions for one more instance with the longer time-limit (instance 21, cf. Geibinger et al. (2019)).

The results for CP Optimizer are particularly interesting. While it only manages to prove the optimality of the 4 smallest instances, it generally finds better solutions for instances which cannot be closed by Chuffed. The fact that CP Optimizer has trouble proving optimality is the main reason why we chose Chuffed for VLNS instead of CP Optimizer.

Simulated Annealing managed to find feasible solutions in all runs but two and most of the time the found solutions are better than the ones found by Chuffed, and competitive to those found by CP Optimizer.

As for VLNS, the search always finds feasible solutions and for 29 of the instances it found a solution with the best known objective. Further, it could prove optimality using the custom lower bounds for 16 of the instances, including two large instances with 60 projects.

Results for the real-world instance can be compared to the reference schedule achieved via manual scheduling. Firstly, this reference schedule omits 90 required resource assignments (88 employee and 2 equipment assignments), and thus does not fulfill all constraints. Disregarding that, it achieves a penalty of 4074 under the same soft constraint weights as for the generated instances. All solvers (except CPLEX and one of the runs of SA) found a feasible solution for this instance in the given time limit. Both CPO, VLNS and SA (in 4 of 5 runs) even managed to find solutions with a markedly lower penalty than the reference schedule. For VLNS, we could show that the best solution found is within 5.97% of the optimum.

6.5 Soft constraints weights

To further evaluate the effect of soft constraint weights, we performed an additional experiment. Starting from a baseline where each objective had a uniform weight of 1, we increased the weight of a single objective to 10000. We did so for each soft constraint and compared the results with the baseline scenario where all weights were set to 1.

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

Table 3 Comparison of different approaches with a timeout of two hours

#	SA			VLNS			Chuffed		CPO		CPLEX	
	Best	#Feas.	Avg	Best		Avg						
1	**98**	5	98	**98**	*	98	**98**	*	**98**	*	**98**	*
2	**73**	5	73	**73**	*	73	**73**	*	**73**	*	**73**	
3	152	5	152.6	**149**	*	149	**149**	*	**149**	*	**149**	
4	106	5	107	**105**	*	105	**105**	*	**105**	*	**105**	*
5	287	5	306.4	**283**		283	**283**	*	**283**		287	
6	180	5	181.4	**162**	*	162	**162**	*	**162**		**162**	
7	**307**	5	307	**307**		307	**307**	*	**307**		–	
8	**310**	5	310	**310**		310	**310**	*	**310**		317	
9	**501**	5	501	**501**	*	501	**501**	*	502		–	
10	**564**	5	564	**564**	*	564	576		566		–	
11	872	5	872.8	**856**		856.4	**856**	*	859		–	
12	660	5	660	**656**	*	656	**656**	*	661		–	
13	352	5	377.4	**340**	*	340	**340**	*	**340**		370	
14	423	5	424.2	**420**	*	420	**420**	*	424		–	
15	1085	5	1086.8	**1084**	*	1085	1646		1104		–	
16	1141	5	1142	**1138**	*	1142.6	1560		1193		–	
17	1195	5	1200.2	**1194**	*	1194.4	1254		1203		1242	
18	**1359**	5	1362.2	**1359**		1378.2	1817		1404		–	
19	2127	5	2168.6	**1964**		2000.8	2650		2099		–	
20	2228	5	2242.2	**2114**		2140.8	2887		2284		–	
21	685	5	689	**679**	*	679.2	**679**	*	722		–	
22	766	5	768.4	**765**		765	**765**	*	770		–	
23	2209	4	2319.25	**2029**		2104.2	3486		2207		–	
24	1780	5	1817.2	**1776**		1798.6	2445		1863		–	
25	2701	5	2836	**2112**		2165.8	3278		2551		–	
26	2579	5	2599.8	**2558**		2573.8	3896		2799		–	
27	2153	5	2155.2	**2151**	*	2151.8	3095		2338		–	
28	2338	5	2351	**2322**	*	2322.6	2569		2402		–	
29	3996	5	4192.6	**3233**		3297	4539		3718		–	
30	**4478**	4	4544.75	4736		4779.4	5904		4995		–	
Lab	3338	4	3366.75	**3334**		3400.6	5188		3444		–	

Column *Best* contains the best solution found, *#Feas.* the number of feasible solutions found (out of 5) and *Avg* the average penalty over all feasible solutions. Solutions for VLNS, Chuffed, CPO and CPLEX marked with "*" indicate that the respective solver proved optimality. The best known solution for each instances is bold

Simulated Annealing and VLNS were run 5 times per instance with different seeds

Figure 3 shows the absolute changes in the penalties s_i ($2 \leq i \leq 5$) when we prioritize a single objective in VLNS. For each instance, we ran VLNS ten times using a run time limit of 30 min. The lines represent the mean of the ten runs, whereas the shadows show the standard deviation.

Setting VLNS to focus on minimizing the number of different employees (shown on the top left in the figure) seems to negatively affect all other soft constraints, especially the project completion times. This is hardly surprising, since minimizing the number of different employees usually involves executing jobs in a serial fashion, thus prolonging project durations. The worsening of the due date violations can be described similarly. For the rise in preferred employee violations, an explanation might be that minimizing the number different employees in a project sometimes means choosing an employee that is not preferred for the job.

For the other soft constraints, the results are less clear. In general, the total project completion time varies significantly between different runs even when the respective soft constraint is given high weight. Furthermore, the impacts the prioritization of a single soft constraints on the other penalties, might also depend on the particular instance but the high variance among the runs makes the drawing of any conclusions difficult.

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

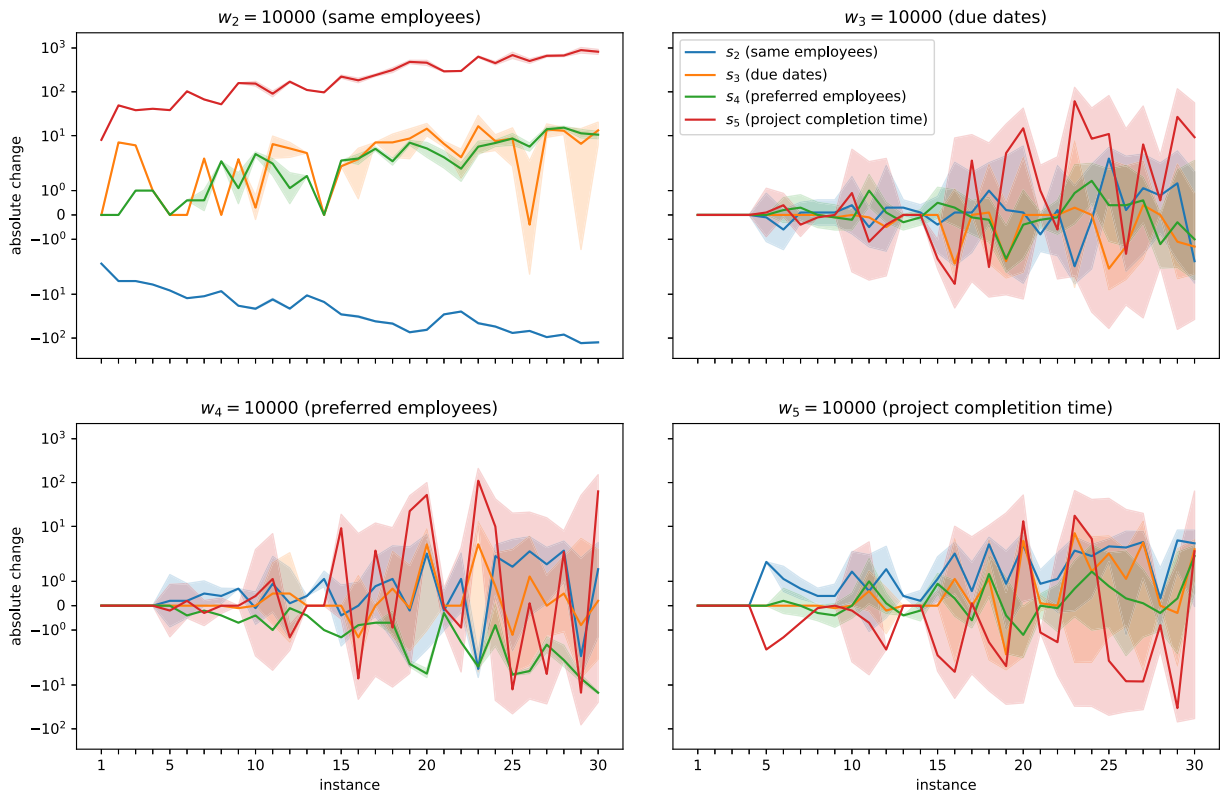

Fig. 3 Changes in penalties when prioritizing a single soft constraint

7 Conclusion

In this paper we have investigated different possibilities to model a complex real-world project scheduling problem. For some of the constraints, we first experimented with approaches which were already used in related project scheduling problems. To deal with this more complex problem and larger instances we introduced several extensions in modeling. Utilising this exact method, we then implemented a VLNS meta-heuristic. We have evaluated our exact approach as well as the VLNS on a set of 30 benchmark instances. Using CP techniques we could find feasible solutions for all considered instances and 15 optimal solutions. Furthermore, the VLNS achieved even better results for most instances and managed to provide 6 provably optimal solutions which could not be found using CP. VLNS also outperformed a local search procedure based on Simulated Annealing and found new best known solutions for several instances, including a real-world instance.

For the future, we plan to investigate exact and hybrid techniques to solve both stages of the TLSP including grouping and scheduling simultaneously.

Acknowledgements The financial support by the Austrian Federal Ministry of Labour and Economy, the National Foundation for Research, Technology and Development and the Christian Doppler Research Association is gratefully acknowledged. The authors acknowledge TU Wien Bibliothek for financial support through its Open Access Funding Programme.

Funding Open access funding provided by TU Wien (TUW).

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

Bartels, J. H., & Zimmermann, J. (2009). Scheduling tests in automotive R&D projects. *European Journal of Operational Research, 193*(3), 805–819. https://doi.org/10.1016/j.ejor.2007.11.010

Bellenguez, O., & Néron, E. (2005). Lower bounds for the multi-skill project scheduling problem with hierarchical levels of skills. In *Proceedings of the 5th international conference on the practice and theory of automated timetabling (PATAT 2005)*, Springer, LNCS,**3616**, 229–243. https://doi.org/10.1007/11593577_14

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

Brucker, P., Drexl, A., Möhring, R., Neumann, K., & Pesch, E. (1999). Resource-constrained project scheduling: Notation, classification, models, and methods. *European Journal of Operational Research, 112*(1), 3–41. https://doi.org/10.1016/S0377-2217(98)00204-5

Chu, G. (2011). Improving combinatorial optimization. PhD thesis, University of Melbourne, Australia, http://hdl.handle.net/11343/36679

Dauzère-Pérès, S., Roux, W., & Lasserre, J. (1998). Multi-resource shop scheduling with resource flexibility. *European Journal of Operational Research, 107*(2), 289–305. https://doi.org/10.1016/S0377-2217(97)00341-X

Demirovic, E., Chu, G., & Stuckey, PJ. (2018). Solution-based phase saving for CP: A value-selection heuristic to simulate local search behavior in complete solvers. In *Proceedings of the 24th international conference on principles and practice of constraint programming (CP 2018)*, Springer, LNCS, **11008**, 99–108. https://doi.org/10.1007/978-3-319-98334-9_7

Drezet, L. E., & Billaut, J. C. (2008). A project scheduling problem with labour constraints and time-dependent activities requirements. *International Journal of Production Economics, 112*(1), 217–225. https://doi.org/10.1016/j.ijpe.2006.08.021

Elmaghraby, S. E. (1977). *Activity networks: Project planning and control by network models*. Wiley.

Feydy, T., Goldwaser, A., Schutt, A., Stuckey, PJ., & Young, KD. (2017). Priority search with minizinc. In *Proceedings of ModRef 2017: The 16th international workshop on constraint modelling and reformulation at CP 2017*, https://ozgurakgun.github.io/ModRef2017/files/ModRef2017_PrioritySearchWithMiniZinc.pdf

Geibinger, T., Mischek, F.,& Musliu, N. (2019). Investigating constraint programming for real world industrial test laboratory scheduling. In *Proceedings of the 16th international conference on the integration of constraint programming, artificial intelligence, and operations research (CPAIOR 2019)*, Springer, LNCS, **11494**, 304–319. https://doi.org/10.1007/978-3-030-19212-9_20

Hartmann, S., & Briskorn, D. (2010). A survey of variants and extensions of the resource-constrained project scheduling problem. *European Journal of Operational Research, 207*(1), 1–14. https://doi.org/10.1016/j.ejor.2009.11.005

IBM, CPLEX (2017a) 12.8.0 IBM ILOG CPLEX Optimization Studio CP Optimizer user's manual. https://www.ibm.com/analytics/cplex-cp-optimizer

IBM, CPLEX (2017b) 12.8.0 IBM ILOG CPLEX Optimization Studio CPLEX user's manual. https://www.ibm.com/analytics/cplex-optimizer

Kirkpatrick, S., Gelatt, C. D., & Vecchi, M. P. (1983). Optimization by simulated annealing. *Science, 220*(4598), 671–680. https://doi.org/10.1126/science.220.4598.671

Laborie, P., Rogerie, J., Shaw, P., & Vilím, P. (2018). IBM ILOG CP optimizer for scheduling. *Constraints, 23*(2), 210–250. https://doi.org/10.1007/s10601-018-9281-x

Mika, M., Waligóra, G., & Węglarz, J. (2015). Overview and state of the art. In C. Schwindt & J. Zimmermann (Eds.), *Handbook on project management and scheduling* (Vol. 1, pp. 445–490). Springer.

Mischek, F., & Musliu, N. (2018a). A local search framework for industrial test laboratory scheduling. In *Proceedings of the 12th international conference on the practice and theory of automated timetabling (PATAT 2018)*, 465–467, https://patatconference.org/patat2018/files/proceedings/paper33.pdf

Mischek, F., & Musliu, N. (2018b). The test laboratory scheduling problem. Technical report, Christian Doppler Laboratory for Artificial Intelligence and Optimization for Planning and Scheduling, TU Wien, CD-TR 2018/1, https://www.dbai.tuwien.ac.at/staff/fmischek/TLSP/TLSP.pdf

Mischek, F., & Musliu, N. (2021). A local search framework for industrial test laboratory scheduling. *Annals of Operations Research, 302*, 533–562. https://doi.org/10.1007/s10479-021-04007-1

Nethercote, N., Stuckey, PJ., Becket, R., Brand, S., Duck, GJ., & Tack, G. (2007). Minizinc: Towards a standard CP modelling language. In *Proceedings of the 13th international conference on principles and practice of constraint programming (CP 2007)*, Springer, LNCS, **4741**, 529–543. https://doi.org/10.1007/978-3-540-74970-7_38

Nudtasomboon, N., & Randhawa, S. U. (1997). Resource-constrained project scheduling with renewable and non-renewable resources and time-resource tradeoffs. *Computers & Industrial Engineering, 32*(1), 227–242. https://doi.org/10.1016/S0360-8352(96)00212-4

Palpant, M., Artigues, C., & Michelon, P. (2004). Lssper: Solving the resource-constrained project scheduling problem with large neighbourhood search. *Annals of Operations Research, 131*(1), 237–257. https://doi.org/10.1023/B:ANOR.0000039521.26237.62

Pritsker, A. A. B., Waiters, L. J., & Wolfe, P. M. (1969). Multiproject scheduling with limited resources: A zero-one programming approach. *Management Science, 16*(1), 93–108. https://doi.org/10.1287/mnsc.16.1.93

Salewski, F., Schirmer, A., & Drexl, A. (1997). Project scheduling under resource and mode identity constraints: Model, complexity, methods, and application. *European Journal of Operational Research, 102*(1), 88–110. https://doi.org/10.1016/S0377-2217(96)00219-6

Schulte, C., Lagerkvist, M., & Tack, G. (2018). Gecode 6.10 reference documentation. https://www.gecode.org

Schwindt, C., & Trautmann, N. (2000). Batch scheduling in process industries: an application of resource-constrained project scheduling. *OR-Spektrum, 22*(4), 501–524. https://doi.org/10.1007/s002910000042

Szeredi, R., & Schutt, A. (2016). Modelling and solving multi-mode resource-constrained project scheduling. In *Proceedings of the 22nd international conference on principles and practice of constraint programming (CP 2016)*, Springer, LNCS, vol 9892, pp 483–492, https://doi.org/10.1007/978-3-319-44953-1_31

Węglarz, J., Józefowska, J., Mika, M., & Waligóra, G. (2011). Project scheduling with finite or infinite number of activity processing modes—A survey. *European Journal of Operational Research, 208*(3), 177–205. https://doi.org/10.1016/j.ejor.2010.03.037

Young, KD., Feydy, T., & Schutt, A. (2017). Constraint programming applied to the multi-skill project scheduling problem. In *Proceedings of the 23rd international conference on principles and practice of constraint programming (CP 2017)*, LNCS, **10416**, 308–317. https://doi.org/10.1007/978-3-319-66158-2_20

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

Terms and Conditions

Springer Nature journal content, brought to you courtesy of Springer Nature Customer Service Center GmbH ("Springer Nature").

Springer Nature supports a reasonable amount of sharing of research papers by authors, subscribers and authorised users ("Users"), for small-scale personal, non-commercial use provided that all copyright, trade and service marks and other proprietary notices are maintained. By accessing, sharing, receiving or otherwise using the Springer Nature journal content you agree to these terms of use ("Terms"). For these purposes, Springer Nature considers academic use (by researchers and students) to be non-commercial.

These Terms are supplementary and will apply in addition to any applicable website terms and conditions, a relevant site licence or a personal subscription. These Terms will prevail over any conflict or ambiguity with regards to the relevant terms, a site licence or a personal subscription (to the extent of the conflict or ambiguity only). For Creative Commons-licensed articles, the terms of the Creative Commons license used will apply.

We collect and use personal data to provide access to the Springer Nature journal content. We may also use these personal data internally within ResearchGate and Springer Nature and as agreed share it, in an anonymised way, for purposes of tracking, analysis and reporting. We will not otherwise disclose your personal data outside the ResearchGate or the Springer Nature group of companies unless we have your permission as detailed in the Privacy Policy.

While Users may use the Springer Nature journal content for small scale, personal non-commercial use, it is important to note that Users may not:

1. use such content for the purpose of providing other users with access on a regular or large scale basis or as a means to circumvent access control;
2. use such content where to do so would be considered a criminal or statutory offence in any jurisdiction, or gives rise to civil liability, or is otherwise unlawful;
3. falsely or misleadingly imply or suggest endorsement, approval , sponsorship, or association unless explicitly agreed to by Springer Nature in writing;
4. use bots or other automated methods to access the content or redirect messages
5. override any security feature or exclusionary protocol; or
6. share the content in order to create substitute for Springer Nature products or services or a systematic database of Springer Nature journal content.

In line with the restriction against commercial use, Springer Nature does not permit the creation of a product or service that creates revenue, royalties, rent or income from our content or its inclusion as part of a paid for service or for other commercial gain. Springer Nature journal content cannot be used for inter-library loans and librarians may not upload Springer Nature journal content on a large scale into their, or any other, institutional repository.

These terms of use are reviewed regularly and may be amended at any time. Springer Nature is not obligated to publish any information or content on this website and may remove it or features or functionality at our sole discretion, at any time with or without notice. Springer Nature may revoke this licence to you at any time and remove access to any copies of the Springer Nature journal content which have been saved.

To the fullest extent permitted by law, Springer Nature makes no warranties, representations or guarantees to Users, either express or implied with respect to the Springer nature journal content and all parties disclaim and waive any implied warranties or warranties imposed by law, including merchantability or fitness for any particular purpose.

Please note that these rights do not automatically extend to content, data or other material published by Springer Nature that may be licensed from third parties.

If you would like to use or distribute our Springer Nature journal content to a wider audience or on a regular basis or in any other manner not expressly permitted by these Terms, please contact Springer Nature at

onlineservice@springernature.com